Docker and Docker Compose

October 27, 2026

Abstract

This report answers the questions on Docker Compose (the problem it solves, how it compares with `docker run`, and how to add a database service) and then summarizes the core Docker concepts: what Docker is and why it matters in data science, the difference between an image and a container, and the purpose of the main Dockerfile instructions.

1 Docker Compose

1.1 What problem does Docker Compose solve?

Real applications rarely consist of a single container. A typical data-science or web project may need an application server, a database, a cache, and perhaps a notebook server. Running each of these by hand with `docker run` means typing long commands with many flags (ports, volumes, environment variables, networks), remembering the correct start-up order, and repeating all of this on every machine.

Docker Compose solves this by letting you *declare the whole multi-container application in one YAML file* (`compose.yaml` or `docker-compose.yml`). Compose then:

- builds or pulls all required images,
- creates a shared network so services can reach each other by service name,
- creates and mounts the volumes,
- starts, stops, and rebuilds everything with a single command.

The result is a reproducible, version-controlled description of the full environment that anyone on the team can start with one command.

1.2 Comparing `docker run` with `docker compose up`

Aspect	`docker run`	`docker compose up`
Scope	Starts *one* container from one image.	Starts *all* services defined in the Compose file.
Configuration	Passed as command-line flags (`-p`, `-v`, `-e`, `-network`, ...).	Declared in a YAML file that can be committed to version control.
Reproducibility	Depends on remembering or scripting the exact command.	The file is the single source of truth.
Networking	Must create and attach networks manually.	A default network is created automatically; services resolve each other by name.
Building images	Requires a separate `docker build` first.	Can build images itself via the `build:` key (`-build` to force).
Dependencies	No notion of start-up order between containers.	`depends_on` (optionally with health checks) controls order.
Lifecycle	Each container is stopped and removed individually.	`docker compose down` stops and removes all services, networks, and optionally volumes.
Typical use	Quick tests, one-off commands, single-container tools.	Multi-service applications and development environments.

For example, the same service started both ways:

```
# With docker run
docker run -d --name app -p 8000:8000 \
  -v $(pwd)/data:/app/data -e ENV=dev my-app:latest

# With Compose (options live in compose.yaml)
docker compose up -d
```

1.3 Extending the Compose project with a database service

Suppose the project has a single `app` service. A database can be added as a second service, here PostgreSQL. The key points are: use an official image, store credentials in environment variables, persist data in a *named volume*, and make the app wait until the database is healthy.

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://user:password@db:5432/mydb
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
```

```
      POSTGRES_DB: mydb
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d mydb"]
      interval: 5s
      timeout: 3s
      retries: 5

volumes:
  db_data:
```

Important details:

- The app reaches the database at hostname `db`, the service name, not `localhost`.
- The named volume `db_data` keeps the data even if the container is removed.
- `depends_on` with `service_healthy` prevents the app from starting before PostgreSQL accepts connections.
- In real projects, secrets should be placed in an `.env` file (excluded from version control) or Docker secrets rather than written directly in the YAML.
- Other databases (MySQL, MongoDB, Redis) are added the same way by changing the image, environment variables, and volume path.

2 Task 5 — Summary Report

2.1 What is Docker, and why is it useful in data-science projects?

Docker is an open-source platform that packages an application together with everything it needs to run (code, runtime, libraries, system tools, and configuration) into a standardized unit that runs the same way on any machine with Docker installed. Unlike a virtual machine, a container shares the host operating-system kernel, so it is lightweight and starts in seconds.

Docker is especially useful in data science because:

- **Reproducibility:** the exact Python/R version and library versions are fixed, so results can be reproduced months later or by a colleague.
- **Dependency isolation:** projects needing conflicting versions of NumPy, TensorFlow, or CUDA can coexist without interfering.
- **Portability:** the same environment runs on a laptop, a server, or in the cloud, which removes the "it works on my machine" problem.
- **Easy collaboration and onboarding:** new team members run one command instead of configuring an environment by hand.
- **Deployment:** a trained model and its serving code can be shipped as a container to production, including orchestration platforms such as Kubernetes.
- **Scalability:** containers are easy to replicate for parallel experiments or serving many requests.

2.2 Difference between a Docker image and a container

	Image	**Container**
Nature	A read-only template made of stacked layers.	A running (or stopped) instance of an image.
State	Static; does not change once built.	Dynamic; has a writable layer, processes, and network.
Created by	`docker build` or `docker pull`.	`docker run` or `docker create`.
Quantity	One image can be reused many times.	Many containers can be started from one image.
Analogy	A class, or a recipe.	An object, or the cooked dish.

In short, the image is the blueprint and the container is the live execution of that blueprint. Changes made inside a container are lost when it is removed unless they are stored in a volume or committed to a new image.

2.3 Purpose of `FROM`, `WORKDIR`, `COPY`, `RUN`, and `CMD`

`FROM` Sets the base image on which the new image is built (for example `python:3.12-slim`). It is normally the first instruction.

`WORKDIR` Sets the working directory inside the image for the instructions that follow and for the running container. It is created if it does not exist.

`COPY` Copies files or folders from the build context on the host (such as `requirements.txt` or source code) into the image file system.

`RUN` Executes a command *at build time* and saves the result as a new layer, typically to install packages (`pip install -r requirements.txt`).

`CMD` Defines the default command executed *when a container starts*. It runs at run-time, not build time, and can be overridden from the `docker run` command line.

A minimal example for a data-science application:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

Copying `requirements.txt` and installing dependencies *before* copying the rest of the code takes advantage of Docker's layer cache: the slow install step is only repeated when the dependencies change.

Conclusion

Docker packages applications and their dependencies into portable images that run as isolated containers, giving data-science projects reproducibility and easy deployment. A Dockerfile de-

fines how an image is built, and Docker Compose builds on this by describing and running multi-container applications, such as an app plus a database, from a single declarative file and a single command.